

Software Engineering Lab Assignment-1

Software Engineering Lab Assignment

Functional and Non-Functional Requirements Analysis

Student Name: Praneesh R V

Team Members:

- Praneesh R V
- Sanjai KCK
- Dinesh V Masuti
- Akash Varunn D

Project Title

A Lightweight, Sidecar-Free Service-to-Service Authentication Framework for Microservices

1. Project Overview

Modern microservice applications commonly rely on service meshes such as Istio and Linkerd to provide service-to-service authentication and authorization. These solutions introduce sidecar proxies that increase memory consumption, latency, and operational complexity.

The proposed project aims to develop a lightweight Go-based middleware framework that enables secure service-to-service authentication and authorization without requiring sidecar proxies. The framework will use SPIFFE/SPIRE-based workload identities and secure delegation of user context between services.

2. Stakeholders

1. Cloud Application Developers
2. DevOps Engineers
3. Security Engineers
4. System Administrators
5. Organizations deploying microservices

6. End Users accessing applications built on the framework

3. Functional Requirements

FR1: Service Identity Verification

The framework shall authenticate every service using SPIFFE-based workload identities before communication is established.

FR2: Mutual Authentication

The framework shall establish mutual TLS (mTLS) between communicating services.

FR3: User Context Delegation

The framework shall securely propagate authenticated user identity information across multiple services.

FR4: Authorization Enforcement

The framework shall evaluate authorization policies before granting access to requested resources.

FR5: Policy Management

The framework shall allow administrators to define and update access control policies.

FR6: Middleware Integration

The framework shall be importable as a Go middleware library and require minimal code changes for integration.

FR7: Request Validation

The framework shall validate incoming service requests and reject unauthorized requests.

FR8: Logging and Auditing

The framework shall maintain logs of authentication, authorization, and access decisions.

FR9: Rogue Service Detection

The framework shall detect and deny requests from services without valid identities.

4. Non-Functional Requirements

NFR1: Performance

Authentication and authorization operations should introduce minimal latency overhead compared to existing service mesh solutions.

NFR2: Low Memory Consumption

The framework should consume significantly less memory than sidecar-based approaches.

NFR3: Scalability

The framework should support deployment across large-scale microservice environments.

NFR4: Reliability

Authentication services should operate consistently without disrupting legitimate service communication.

NFR5: Security

All communications must be encrypted using industry-standard cryptographic protocols.

NFR6: Availability

The framework should remain operational even under high request loads.

NFR7: Maintainability

The codebase should be modular, documented, and easy to maintain.

NFR8: Portability

The framework should function across different deployment environments and cloud platforms.

NFR9: Usability

Developers should be able to integrate the framework with minimal configuration effort.

5. Constraints

1. Development language must be Go.
 2. SPIFFE/SPIRE infrastructure is required for workload identity management.
 3. Testing environment will use Docker and Kubernetes (Minikube).
 4. Limited hardware resources available for development and testing.
 5. Project timeline restricted to the academic semester duration.
-

6. Assumptions

1. Services participating in communication possess valid SPIFFE identities.
 2. Network connectivity exists between communicating services.
 3. Administrators define authorization policies correctly.
 4. Services support HTTPS communication.
-

7. Requirement Elicitation Process

The requirements were identified through:

- Literature survey on service mesh security solutions.
 - Analysis of existing frameworks such as Istio, Linkerd, Ambient Mesh, and SPIFFE.
 - Study of documented security incidents including the Tesla Kubernetes Cryptojacking attack.
 - Team discussions on performance, security, and deployment challenges in modern microservice architectures.
-

8. Individual Contribution of Team Members in Requirement Identification

Team Member	Contribution
Sanjai K C K	Identified core problem statement, security requirements, SPIFFE/SPIRE integration requirements, authentication and authorization requirements, and threat analysis based on literature survey.
Praneesh R V	Contributed to identifying performance-related requirements including latency reduction, memory optimization, scalability, and benchmarking requirements.

Team Member	Contribution
Dinesh V Masuti	Contributed to deployment, portability, reliability, and maintainability requirements. Assisted in defining system constraints and assumptions.
Akash Varunn D	Contributed to policy management, logging, auditing, usability, compatibility, and extensibility requirements. Assisted in stakeholder analysis.

9. Conclusion

The proposed framework aims to provide secure, lightweight, and platform-independent service-to-service authentication for microservices without relying on sidecar proxies. The identified functional and non-functional requirements establish the foundation for system design, implementation, testing, and evaluation during subsequent development phases.
